

1. Facade pattern

Facade pattern je veoma koristan kada je potrebno kontrolisati koje funkcionalnosti su vidljive. U ovom slucaju, klijent sistem mora direktno koristiti metode za internu logiku da bi izvrsio neke funkcionalnosti. Umjesto toga, moze se napraviti SistemFacade sa „omotavajucim“ metodama potrebnim sistemu, koje ce sakriti i koristiti interne metode radi jednostavnijeg prikaza. Npr. Prijavilspit automatski koristi metodu za provjeru roka bez da klijent to mora.

Metode:

bool login(String lozinka, String email) – koristi provjeriLozinku

void Prijavilspit(int indeks, int ispitId) – koristi dodajPrijavu, provjeriRokZaPrijavu

void PredajZadacu(int indeks, int predajaZadace.Id, string fajl) – koristi dodajPredajuZadace

void unesiBodovanje(int indeks, int Profesor.id, int sifraPredmeta, PredajaZadace predaja, int bodovi)

void unesiOcjenu(int Profesor.id, int indeks, int sifraPredmeta, int ocjena)

void obradiZahtjev(int idZahtjeva)

2. Adapter pattern

Moze se koristiti za generisanje dokumenata, npr. DokumentAdapter koji ce ZahtjevZaDokument pretvoriti u format koji vanjski servis moze koristiti. Sistem ce pozivati metodu, a ona ce interno pozivati vanjski sistem bez da sistem mora pretvarati formate.

3. Proxy pattern

Protection proxy moze vrsiti razne provjere ispravnosti i dozvole pristupa, npr da li student pokusava provjeriti zahtjev za dokument, zadacu ili prijavu ispita poslanu od strane drugog studenta. ZahtjevProxy bi imao isti interfejs kao ZahtjevZaDokument, ali bi svaku metodu cuvao provjerom uloge — Student može samo vidjeti svoje

zahtjeve, a StudentskaSluzba može obrađivati sve. Klijent ne zna da razgovara s proxyjem, misli da direktno koristi ZahtjevZaDokument

Virtual proxy može biti koristen kod ZahtjevZaDokument, odgadja kreiranje zahtjeva sve dok student ne uputi zahtjev. Kreiranje ZahtjevZaDokument objekta je skupo pa se odgađa do trenutka stvarnog slanja zahtjeva.

4. Composite pattern

Izracun interfejs može biti koristen za cesto koristene operacije u klasama Ispit, Zadaca i Predmet. Može imati metode izracunajProsjeck, getStatus, getBodovi koje su potrebne sve tri klase. Konkretno, Predmet sadrži listu Ispit-a i listu Zadaca-a.

Composite omogućava da se izracunajProsjeck() poziva na Predmet-u, koji onda delegira na sve Ispit i Zadaca objekte uniformno.

Student bi mogao imati listu Predmet-a i računati ukupni prosjek rekurzivno

5. Decorate pattern

Bio bi veoma koristan u slucaju da se za razlicite dokumente koje treba generisati trebaju dopuniti neke funkcionalnosti npr. Dokument sa ili bez pecata, ali kako svaki dokument treba pecat, i kako dva dokumenta koja se mogu generisati su potvrda o studiranju i prepis ocjena, a oni su dva razlicita tipa umjesto da je jedan dopuna drugog, nema svrhe koristiti decorator pattern tu

6. Bridge pattern

Veoma koristan kod slanja notifikacija. Kako korisnik nasljedjuju student, profesor i studentska sluzba, i svaka klasa zahtijeva razlicite notifikacije, a i mogu se slati putem emaila i u aplikaciji, bridge pattern pomaze da se smanji broj potrebnih klasa. Apstrakcija: Korisnik sa metodom posaljiObavijest()

Interfejs: Notifikacije sa metodama posalji(poruka)

Konkretni: EmailNotif, InAppNotif